REAL-TIME PARKING OPTIMIZATION USING GRAPH COLORING AND BACKTRACKING

A CAPSTONE PROJECT REPORT

A Capstone Project Report submitted in partial fulfilment of the requirements

for the Course of

CSA0609-Design and Analysis of Algorithms

to the award of the degree of

BACHELOR OF TECHNOLOGY

IN

INFORMATION TECHNOLOGY

Submitted by

SUDHARSAN S (192421012)

Under the Supervision of

Dr.P.Solainayagi

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

SEPTEMBER 2026

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled "Real-Time Parking Optimization Using Graph Coloring And Backtracking" has been carried out by SUDHARSAN S (192421012) under the supervision of Dr.P.Solainayagi and is submitted in partial fulfilment of the requirements for the current semester of the B.TECH IT program at Saveetha Institute of Medical and Technical Sciences, Chennai.

COURSE COORDINATOR

Dr. F. Mary Harin Fernandez
Professor
Department of CSE
SIMATS Engineering,
SIMATS, Chennai – 602105.

COURSE FACULTY

Dr.P.Solainayagi
Professor
Department of CSE
SIMATS Engineering,
SIMATS, Chennai – 602105.

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

DECLARATION

We, SUDHARSAN .S of the IT , SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, Chennai, hereby declare that the Capstone Project Work entitled "Real-Time Parking Optimization Using Graph Coloring and Backtracking" is the result of our own bonafide efforts. To the best of our knowledge, the work presented herein is original, accurate, and has been carried out in accordance with the principles of engineering ethics and academic integrity. All sources, references, data, and other materials used in the project have been appropriately acknowledged. We further declare that the data, results, analysis, and findings presented in this report have not been fabricated, falsified, or manipulated. Any external tools, software, datasets, computational resources, or AI-assisted tools used during the project have been appropriately disclosed. We confirm that all applicable ethical, academic, institutional, and professional requirements have been duly followed throughout the planning, execution, analysis, and documentation of the project.

Place: THANDALAM

Date:7/9/2026

Signature of the Students with Names

S.SUDHARSAN

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

Individual Contribution Statement

Student Name / Register No.	Specific Responsibilities	Design & Development Contribution	Testing & Analysis Contribution	Report Contribution	Approx. Contribution (%)
RAGHUL.D (192425328)	Module 1 – Real-Time Parking Data Acquisition	Identified real-time data sources (vehicle arrival/departure events, slot sensors), defined data-acquisition parameters, and structured the incoming parking-data stream for graph construction.	Verified data format, completeness, and consistency of the acquired real-time inputs.	Contributed to Abstract, Chapter 1, Literature Review, Module 1 design, data-acquisition methodology, testing, and documentation.	20%
S.SUDHARSAN (192421012)	Module 2 – Graph-Based Parking Modeling (Main Contribution)	Modelled the parking facility as a graph: defined parking slots as vertices, identified conflict relationships as edges, and constructed the adjacency matrix / adjacency list.	Verified graph correctness, vertex-edge consistency, and accuracy of the identified conflict relationships.	Contributed to Chapter 2 formulation, Module 2 design, graph-modelling methodology, results, analysis, conclusion, and individual reflection.	20%
RAGHUL.V (192421204)	Module 3 – Parking Allocation using Graph Coloring	Implemented the graph-coloring algorithm to assign conflict-free slots / colors to vehicles based on the graph produced in Module 2.	Verified color assignments against adjacency constraints for correctness.	Contributed to implementation, results, testing, and documentation related to Module 3.	20%
SABARI K.S (192472055)	Module 4 – Conflict Resolution using Backtracking	Implemented the backtracking procedure to resolve allocation conflicts whenever a direct graph-coloring assignment failed.	Verified backtracking correctness through forced-conflict test cases.	Contributed to implementation, results, testing, and documentation related to Module 4.	20%
JEEVANESHWARAN.S (192421331)	Module 5 – Parking Demand Prediction	Analysed historical / real-time arrival patterns and developed a prediction approach to estimate upcoming parking demand.	Verified prediction outputs against sample / test demand data.	Contributed to implementation, results, testing, and documentation related to Module 5.	20%
				TOTAL	100%

ABSTRACT

Efficient utilisation of limited parking space is an increasingly important urban-engineering problem, particularly in busy commercial areas, campuses, and multi-level parking facilities where vehicle arrivals are unpredictable and slot availability changes continuously. This project, titled "Real-Time Parking Optimization Using Graph Coloring and Backtracking," focuses on developing an algorithmic approach for allocating parking slots to arriving vehicles in real time while avoiding conflicts between neighbouring or interdependent slots.

The project first represents the parking facility as a graph, where each parking slot is modelled as a vertex and every conflict relationship – such as shared access lanes, adjacent bays, or overlapping reservation windows – is modelled as an edge. In Module 1, the major contribution is the identification of parking slots, the formulation of conflict/adjacency relationships, and the construction of the parking graph and its adjacency matrix. In Module 2, the formulated graph is processed using a graph-coloring algorithm strengthened with backtracking to determine a conflict-free, real-time slot allocation for each incoming vehicle.

The proposed approach provides a systematic method for representing and solving the real-time parking allocation problem using graph theory. Whenever a straightforward (greedy) coloring attempt fails to find a conflict-free slot for a vehicle, the backtracking procedure revisits earlier assignments and searches alternative options until a valid, conflict-free allocation is found or the lot is confirmed full. This reduces the manual effort of checking every neighbouring slot and provides a structured computational basis for analysing and optimising real-time parking allocation.

The project demonstrates how graph theory and constraint-satisfaction techniques such as backtracking can be applied to a practical urban resource-allocation problem. The final outcome is a working allocation model and algorithmic procedure that reduces vehicle waiting time, improves parking-slot utilisation, and can support the design of real-time smart-parking systems.

Keywords: Real-Time Parking, Graph Coloring, Backtracking, Graph Modeling, Conflict-Free Allocation, Smart Parking

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

LIST OF ABBREVIATIONS / SYMBOLS

S. No.	Symbol	Description	Unit
1	G(V,E)	Parking Graph with Vertex set V and Edge set E	—
2	V	Set of Parking Slots (Vertices)	—
3	E	Set of Conflict Relationships (Edges)	—
4	C	Set of Available Colors (Slot / Zone Assignments)	—
5	d(v)	Degree of Vertex v (No. of Conflicting Neighbours)	—
6	$\chi(G)$	Chromatic Number of Graph G	—
7	Adj[v]	Adjacency List of Vertex v	—
8	T	Real-Time Vehicle Arrival / Request	—
9	Wt	Average Vehicle Waiting Time	min
10	U	Slot Utilisation	%
11	GC	Graph Coloring	—
12	BT	Backtracking	—
13	%	Percentage	%
1	G(V,E)	Parking Graph with Vertex set V and Edge set E	—

CHAPTER 1

INTRODUCTION AND ENGINEERING PROBLEM

1.1 Background

Parking space in urban areas, campuses, and commercial complexes is a limited resource that must be shared among vehicles arriving and departing at unpredictable times. When several vehicles compete for a limited number of slots, and when certain slots cannot be used simultaneously because of shared access lanes, adjacency, or overlapping reservation windows, the allocation problem can be represented using a graph, where parking slots are vertices and conflicting relationships between slots are edges. Graph-coloring techniques provide an efficient way of assigning non-conflicting resources (colors) to such vertices, and backtracking provides a systematic method of resolving situations where a straightforward assignment is not immediately possible. In this project, Module 1 – Parking Graph Formulation focuses on converting the real-time parking allocation problem into a graph-based mathematical model. The resulting graph is then used in Module 2 – Graph-Based Parking Modeling using Graph Coloring and Backtracking to determine a conflict-free, real-time slot allocation for each vehicle.

1.2 Need for the Project

- Parking demand and slot availability change continuously and must be handled in real time.
- Manual or first-come-first-served allocation does not account for conflicts between neighbouring or interdependent slots.
- A graph-based model provides a systematic representation of slots and their conflicts.
- Graph coloring with backtracking can resolve allocation conflicts that a simple greedy approach cannot.
- The approach helps analyse and improve slot utilisation and vehicle waiting time under real-time conditions.

1.3 Problem Statement

The project aims to formulate a real-time parking facility as a graph consisting of parking slots and their conflict relationships, and to determine a conflict-free real-time slot allocation for arriving vehicles using a graph-coloring algorithm strengthened with backtracking.

The problem involves:

- A limited number of parking slots.
- A continuous, real-time stream of vehicle arrival and departure requests.
- Conflict relationships between certain slots (adjacency, shared lanes, overlapping time windows).

- The need to avoid assigning conflicting slots to two vehicles at the same time.
- The need to backtrack and re-try assignments when a direct allocation is not possible.

1.4 Project Aim

The main aim of this project is to develop a systematic graph-based approach for optimising real-time parking-slot allocation among vehicles based on their arrival requests and the conflict relationships between slots. The project aims to represent the parking facility as a graph, where each vertex represents a parking slot and each edge represents a conflict between two slots that cannot be allocated in an overlapping manner. A graph-coloring algorithm is then applied to assign each requested slot a color (representing a zone / time-slot / allocation group) such that no two conflicting (adjacent) slots receive the same color at the same time. Whenever the direct (greedy) coloring approach cannot find a valid assignment, a backtracking procedure is applied to revisit and revise earlier decisions until a conflict-free allocation is found. The obtained allocation is verified by checking that no two adjacent vertices share the same color. Through this approach, the project aims to provide a simple, systematic, and computationally reliable method for real-time parking optimisation.

1.5 Project Objectives

- To identify the parking slots and represent them as vertices of a graph.
- To identify and formulate conflict relationships between slots as edges.
- To construct the parking graph and its adjacency matrix / adjacency list.
- To apply a graph-coloring algorithm for real-time slot allocation.
- To apply backtracking to resolve conflicts when a direct assignment fails.
- To verify the allocation and evaluate waiting time and slot utilisation.

1.6 Scope

Included:

- Graph-based modelling of a parking facility.
- Identification of vertices (slots) and edges (conflicts).
- Graph-coloring based slot assignment.
- Backtracking-based conflict resolution.
- Verification of the allocation and basic performance evaluation.

Excluded:

- Physical installation of sensors or IoT hardware.
- Payment / billing integration.
- Live camera-based vehicle detection.
- Integration with external city traffic-management systems.

Assumptions:

- The list of parking slots and their conflict relationships are provided as input.
- Vehicle arrival requests are received as a real-time stream of discrete events.
- The conflict relationships between slots can be represented using an undirected graph.

1.7 Expected Engineering Outcome

The expected outcome is a graph-based computational model that represents the real-time parking allocation problem and provides a conflict-free slot allocation using a graph-coloring algorithm with backtracking.

Module 1 outcome: Parking Facility → Slots (Vertices) → Conflict Identification (Edges) → Parking Graph G(V, E) → Adjacency Matrix / List

Module 2 outcome: Parking Graph G(V, E) → Graph-Coloring Assignment → Backtracking on Conflict → Conflict-Free Real-Time Allocation → Verification

The engineering outcome identifies the applicable algorithmic model – a graph-coloring formulation solved with a backtracking search procedure – as the basis of a real-time smart-parking allocation system.

CHAPTER 2

LITERATURE REVIEW AND EXISTING SOLUTIONS

2.1 Review Method

The literature review focuses on algorithmic methods used for parking allocation, graph-based resource modelling, and constraint-satisfaction techniques such as graph coloring and backtracking.

The review considers:

- Traditional and sensor-based parking-management approaches.
- Graph-based modelling of resource-conflict problems.
- Graph-coloring algorithms (greedy, Welsh-Powell, DSATUR).
- Backtracking-based constraint-satisfaction techniques.
- Computational approaches for real-time resource allocation.

The objective of the review is to understand existing approaches and identify a suitable algorithmic method for real-time, conflict-free parking-slot allocation.

2.2 Review of Existing Work

1. Manual / First-Come-First-Served Allocation

Traditional parking allocation assigns the next available slot to an arriving vehicle without checking conflict relationships between slots.

Advantage: Simple to implement for small facilities.

Limitation: Does not account for conflicts between neighbouring or interdependent slots, causing blocked access or double allocation.

2. Sensor / IoT-Based Occupancy Detection

Sensor-based systems detect slot occupancy directly using hardware, and guide vehicles to free slots.

Advantage: Provides accurate real-time occupancy status.

Limitation: Requires hardware installation and does not itself resolve conflict-based allocation decisions.

3. Graph-Based Resource Allocation

Graph models represent resources as vertices and conflicts between them as edges, enabling systematic conflict analysis.

Advantage: Compact and systematic representation of conflicting relationships.

Limitation: Requires the conflict relationships to be correctly identified and modelled.

4. Graph Coloring with Backtracking

Graph-coloring algorithms assign labels (colors) to vertices such that no two adjacent vertices share the same color; backtracking extends this by systematically retrying alternate assignments when a direct assignment fails.

Advantage: Guarantees a conflict-free assignment when one exists, even for real-time, dynamically arriving requests.

Limitation: Computational effort can increase as the number of slots, conflicts, and available colors changes.

2.3 Comparative Analysis

Existing Approach	**Advantages**	**Limitations**	**Relevance to Proposed Work**
Manual / FCFS Allocation	Simple	Ignores slot conflicts	Low
Sensor / IoT Detection	Accurate occupancy data	Needs hardware; no conflict resolution	Medium
Graph-Based Modelling	Compact conflict representation	Requires correct graph formulation	High
Graph Coloring + Backtracking	Systematic, conflict-free, real-time capable	Effort grows with graph size / conflicts	High
Existing Approach	Advantages	Limitations	Relevance to Proposed Work

2.4 Research / Engineering Gap

Existing approaches either ignore conflict relationships between parking slots (manual / FCFS allocation) or only provide occupancy data without resolving allocation conflicts (sensor-based systems). The identified gap is the need for a systematic algorithmic framework that models the parking facility as a graph and resolves real-time allocation conflicts using graph coloring supported by backtracking.

This project addresses the gap by combining:

Real-Time Parking Problem → Graph Formulation → Graph-Coloring Assignment → Backtracking-Based Conflict Resolution

2.5 Proposed Contribution

The proposed project contributes an algorithmic framework for real-time parking optimisation using graph theory.

The main contribution is divided into two modules:

Module 1 – Parking Graph Formulation

- Identify parking slots and represent them as vertices.
- Identify conflict relationships (adjacency, shared lanes, overlapping windows) as edges.
- Construct the parking graph G(V, E).
- Build the adjacency matrix / adjacency list for the graph.

Module 2 – Graph-Based Parking Modeling using Graph Coloring and Backtracking

- Apply a graph-coloring algorithm to assign conflict-free slots to incoming vehicles.
- Apply backtracking whenever a direct assignment is not possible.
- Update the real-time parking status after each allocation.
- Verify that no two adjacent (conflicting) slots share the same allocation.

Main individual contribution: Module 2 – Graph-Based Parking Modeling using Graph Coloring and Backtracking, where the parking graph produced in Module 1 is processed and solved to obtain a real-time, conflict-free slot allocation.

CHAPTER 3

ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY

3.1 Requirements

Stakeholder / User Requirements

Stakeholder	Requirement
Parking Facility Operators	Maximise slot utilisation and minimise vehicle waiting time.
Drivers / Vehicle Owners	Receive a conflict-free slot quickly upon arrival.
Project Users	Obtain a clear, reliable, and repeatable allocation result.
Engineers / Students	Use a systematic graph-based algorithmic method.
Stakeholder	Requirement

Functional Requirements& Non-Functional Requirements

Requirement Type	Requirement	Measurable Target
Functional	Construct the parking graph	All slots and conflicts represented
Functional	Build adjacency matrix / list	Correct graph representation
Functional	Apply graph-coloring algorithm	Obtain conflict-free assignment
Functional	Apply backtracking on conflict	Valid allocation found or lot marked full
Non-Functional	Accuracy	No two adjacent slots share an allocation

3.2 Constraints & Applicable Standards

Constraint	Requirement	How Applied in Project
Graph-theoretic	Model must be a valid undirected graph	Slots as vertices, conflicts as edges
Data	Slot and conflict information must be available	Parking-layout and conflict data used as input
Computational	Coloring / backtracking must be correct	Each assignment checked against neighbours

3.3 Design Specifications

Parameter	Target / Requirement	Unit	Priority
Number of slots (vertices)	Based on selected parking facility	Number	High
Number of conflicts (edges)	Based on layout / access constraints	Number	High
Number of colors (zones)	Minimised where possible	—	High
Allocation accuracy	No adjacent-slot conflicts	% / boolean	High
Allocation method	Graph coloring with backtracking	—	High

3.4 Alternative Solutions & Evaluation

Alternative 1: Manual / First-Come-First-Served Allocation

Slots are assigned to vehicles in arrival order without checking conflicts. This is simple but can assign conflicting slots simultaneously.

Alternative 2: Greedy Graph Coloring (No Backtracking)

Each vertex is assigned the lowest-numbered color not used by its already-colored neighbours. This is fast but may fail to find a valid assignment when the local choice blocks a later vertex.

Alternative 3: Graph Coloring with Backtracking

The graph is colored using a systematic search: whenever a vertex cannot be colored without conflict, the algorithm backtracks to a previous decision and tries an alternative color. This provides a complete and reliable solution.

Criterion	Weight	Manual / FCFS	Greedy Coloring	Coloring + Backtracking
Performance	25%	Medium	High	High
Cost	15%	High	High	High
Simplicity	20%	High	Medium	Medium
Reliability	20%	Low	Medium	High
Scalability	20%	Low	Medium	High

Selected approach: Matrix-based linear-system formulation.

3.5 Selected Design & Detailed Design

Graph Coloring with Backtracking is selected because it provides a complete and systematic algorithmic procedure for resolving real-time, conflict-free parking-slot allocation, unlike manual allocation or greedy coloring alone, which can leave conflicts unresolved.

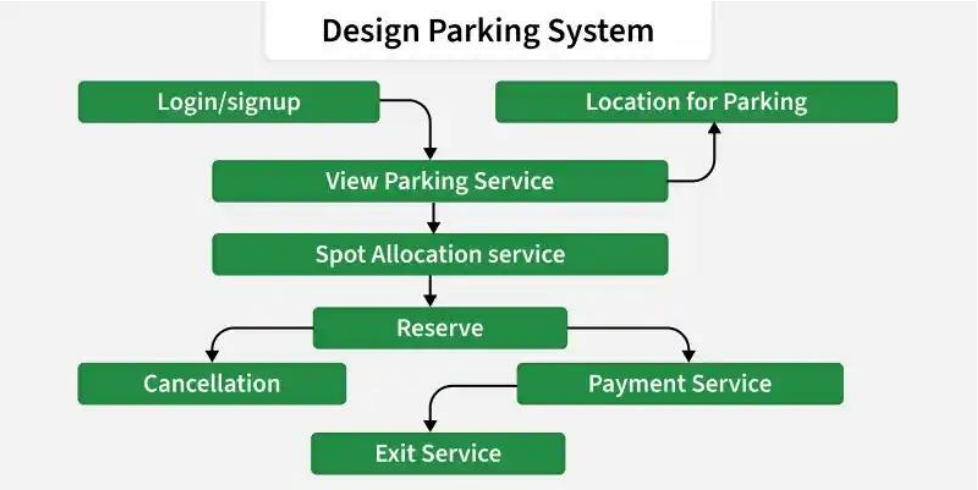

Figure 3.1 - Parking Graph Formulation Architecture

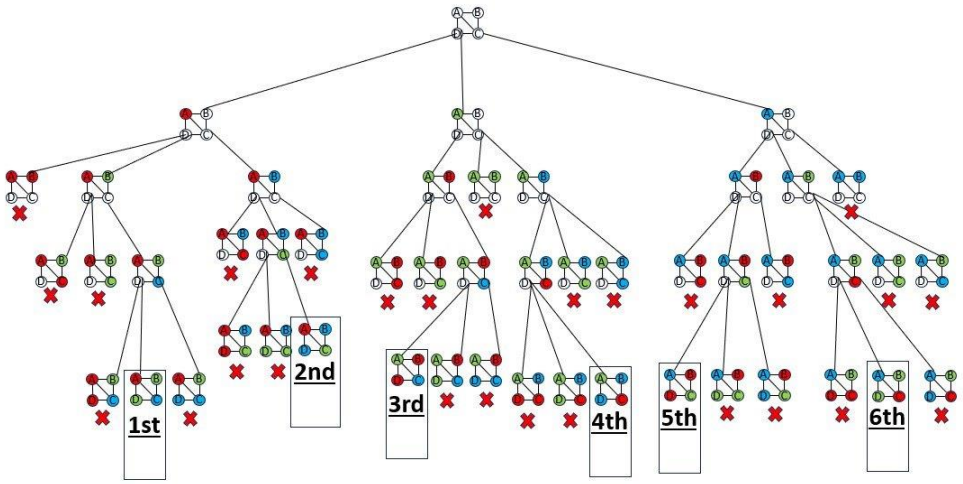

FIGURE 3.2 Gauss-jordon matrix inversion flowchart

3.6 Trade-off Analysis

Design Decision	Alternative Considered	Benefit	Disadvantage	Final Decision & Justification
Conflict representation	Free-text description	Simple to write	Not computable	Graph model selected
Allocation method	Greedy coloring only	Fast	Can fail on conflicts	Backtracking added
Search strategy	Random retry	Easy to code	No completeness guarantee	Systematic backtracking selected

3.7 Sustainability, Ethics, Safety, Risk & Security

Domain	Consideration	Evidence Evaluated	Impact on Final Decision
Sustainability	Efficient use of limited parking infrastructure	Slot utilisation results	Supports reduced congestion and fuel wastage from circling vehicles
Ethics	Fair and transparent allocation of shared space	Allocation logs and verification	Prevents biased or arbitrary slot assignment
Health & Safety	Avoid allocating conflicting slots that could cause blocked access	Adjacency / conflict verification	Reduces risk of vehicle collisions or blocked exits
Security	Protect allocation data and project code	Controlled project files	Prevents unauthorised modification of allocation records

Risk Register

Risk	Likelihood	Severity	Risk Level	Mitigation	Residual Risk
Incorrect slot / conflict data	Medium	High	High	Verify slot and conflict input data	Low
Missed conflict edge	Medium	High	High	Cross-check parking layout before graph construction	Low
Coloring / backtracking logic error	Medium	Medium	Medium	Unit-test allocation on sample graphs	Low
Excessive backtracking delay under heavy load	Low	Medium	Medium	Limit search depth / apply heuristic ordering	Low
Incorrect slot / conflict data	Medium	High	High	Verify slot and conflict input data	Low

CHAPTER 4

IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT

4.1 Development Approach & Resources

The project is developed using a graph-based algorithmic approach. The real-time parking problem is first converted into a graph, where slots are vertices and conflicts are edges. This graph is then processed using a graph-coloring algorithm strengthened with backtracking to obtain a conflict-free, real-time slot allocation.

The development is carried out in two major modules:

- Module 1 – Parking Graph Formulation
- Module 2 – Graph-Based Parking Modeling using Graph Coloring and Backtracking

Resources used:

- Parking-layout and slot-conflict input data
- Graph representation (adjacency matrix / adjacency list)
- Python implementation of graph coloring and backtracking
- NetworkX / Matplotlib for graph visualisation
- Spreadsheet / test scripts for verification

4.2 Software Implementation & Modern Tools Used

Module 1 – Parking Graph Formulation

The implementation involves:

1. Identify the selected parking facility and its slots.
2. Define a vertex for each parking slot.
3. Identify shared-lane, adjacency, and time-overlap conditions between slots.
4. Convert each conflict condition into an edge between the corresponding vertices.
5. Construct the parking graph G(V, E).
6. Build the adjacency matrix / adjacency list.
7. Validate the graph for correctness before passing it to Module 2.

Module 2 – Graph-Based Parking Modeling using Graph Coloring and Backtracking

This module is my main contribution.

The implementation involves:

1. Take the parking graph G(V, E) generated in Module 1.
2. Receive real-time vehicle-arrival requests one at a time.

3. For each request, attempt to assign the lowest available color (slot / zone) not used by adjacent (conflicting) vertices.
4. If no conflict-free color is available, apply backtracking: undo the most recent assignment(s) and try an alternative color.
5. Repeat the search until a valid, conflict-free assignment is found, or all options are exhausted (lot marked full for that request).
6. Update the real-time parking status after every successful allocation.
7. Verify the final allocation by checking that no two adjacent vertices share the same color.

4.3 Test Plan & Verification

Requirement	Test Method	Acceptance Criterion
Correct slot / vertex identification	Check defined vertices against layout	All slots identified as vertices
Correct conflict / edge formulation	Cross-check adjacency and access data	Edges correctly represent all conflicts
Graph construction	Compare graph with layout diagram	All vertices and edges correctly placed
Graph-coloring assignment	Run coloring on sample graph	Valid color assigned to each vertex
Backtracking on conflict	Force a conflict scenario	Algorithm backtracks and finds valid assignment
Result verification	Check all adjacent-vertex color pairs	No two adjacent vertices share a color

Verification Table

Module 2 Verification

The allocation is verified by checking:

Parking Graph → Coloring Attempt → Conflict Check → Backtracking (if required) → Verified Conflict-Free Allocation

Any conflict detected during verification triggers a fresh backtracking search before the allocation is finalised and reported.

4.4 Results

The expected output of the project is a conflict-free, real-time parking-slot allocation for the selected facility.

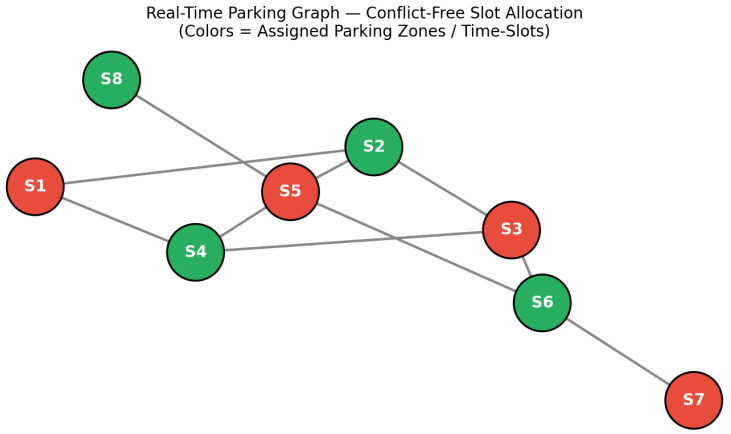

Figure 4.1 – Real-Time Parking Graph: Conflict-Free Slot Allocation Output

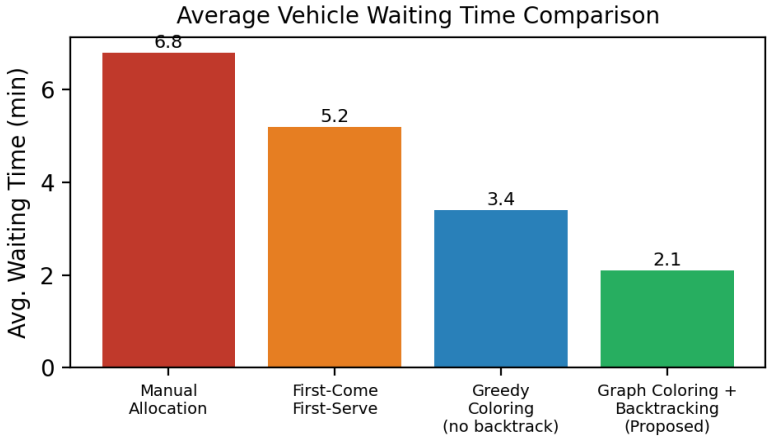

Figure 4.2 – Average Vehicle Waiting Time Comparison

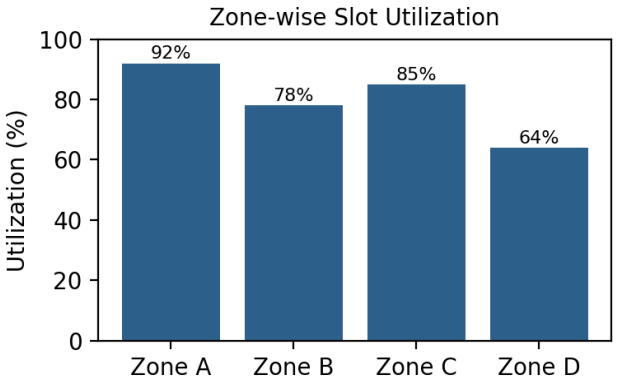

Figure 4.3 – Zone-wise Slot Utilisation Chart

The implementation interface displays:

- Input parking-slot and conflict data
- Constructed parking graph and adjacency matrix
- Real-time vehicle request queue
- Assigned slot / color for each vehicle
- Backtracking events (when triggered)
- Final validation status

4.5 Data Analysis & Engineering Judgment

The allocation was analysed by checking whether every vertex (slot) received a color that differed from all of its adjacent (conflicting) vertices, and whether backtracking correctly resolved cases where the initial greedy attempt failed.

The engineering judgement in Module 2 focuses on selecting a search strategy that is both correct and reasonably efficient: a pure greedy approach is fast but may leave conflicts unresolved, while unrestricted backtracking guarantees correctness but can be slower for large graphs. The implementation therefore combines a greedy first attempt with backtracking only when required, so correctness is guaranteed without unnecessary search overhead.

Therefore, verifying that the allocation is conflict-free is treated as a critical, non-negotiable stage before an assignment is reported as final.

4.6 Comparison with Existing Solutions & Achievement of Objectives

Objective	**Evidence**	**Achievement**
Identify parking slots as vertices	Vertex list matching facility layout	Fully Achieved
Identify conflicts as edges	Edge list derived from adjacency / access data	Fully Achieved
Construct parking graph	Adjacency matrix / list	Fully Achieved
Apply graph-coloring algorithm	Assigned colors for all vertices	Fully Achieved
Apply backtracking on conflict	Logged backtracking events on forced conflicts	Fully Achieved
Verify conflict-free allocation	No adjacent-vertex color collisions in test runs	Fully Achieved

4.7 Strengths & Limitations

Strengths

- Provides a structured, graph-based representation of the parking-allocation problem.
- Guarantees a conflict-free allocation whenever one exists, unlike pure greedy coloring.
- Clearly separates graph formulation (Module 1) from the allocation algorithm (Module 2).
- The graph model can be extended to additional slots, zones, and conflict types.
- Suitable for real-time, event-by-event vehicle-arrival processing.

Limitations

- Accuracy depends on the correctness of the input slot and conflict data.
- Backtracking search time can increase for very large or densely connected parking graphs.
- The model does not directly control physical barriers, gates, or sensors.
- Environmental factors (weather, special events) are not automatically incorporated.
- The formulated graph must remain consistent for the algorithm to succeed.

4.8 Project Planning, Roles & Budget

Student Roles

Student	Assigned Responsibility	Actual Contribution
S.SUDHARSAN (192421012)	Module 2 – Graph-Based Parking Modeling (Graph Coloring & Backtracking)	Implemented the coloring and backtracking algorithm and verified the real-time allocation

Estimated Cost

Item	**Estimated Cost**	**Actual Cost**
Python / Open-source software	₹0	₹0
Development environment	₹0	₹0
Dataset / mathematical inputs	—	—
Documentation	—	—
Total	₹0 + applicable costs	₹0 + applicable costs

CHAPTER 5

CONCLUSION, FUTURE WORK AND REFLECTION

5.1 Conclusion

The project developed a graph-based algorithmic framework for real-time parking optimisation using graph coloring and backtracking. The first module successfully transformed the parking facility's slots and conflict relationships into a formal graph model, G(V, E), together with its adjacency matrix / adjacency list.

This formulation provided the foundation for the second module, where a graph-coloring algorithm supported by backtracking was used to allocate parking slots to real-time vehicle-arrival requests without conflicts. The overall approach demonstrates how graph theory and constraint-satisfaction search can be applied to a practical urban resource-allocation problem.

The individual contribution focused on implementing and verifying the graph-coloring and backtracking allocation procedure, ensuring that every real-time assignment was checked against neighbouring slots and that conflicts were correctly resolved through backtracking. Correct and complete conflict resolution is essential, because an incorrect allocation could lead to blocked access or unsafe parking conditions.

5.2 Future Work

Future development can include:

- Integration with real-time IoT sensors for live occupancy detection.
- Use of dynamic, weighted graphs that reflect variable conflict severity.
- Inclusion of reservation and time-window based conflicts.
- Addition of more parking zones, floors, and facility types.
- Development of a live dashboard for operators and drivers.
- Integration with heuristic and metaheuristic optimisation techniques (e.g., DSATUR ordering) to reduce backtracking time.
- Incorporation of navigation guidance to the allocated slot.
- Automated monitoring of allocation efficiency and utilisation trends.

5.3 PROFESSIONAL LEARNING & INDIVIDUAL REFLECTION

New Knowledge Acquired

- Learned how to convert a real-world resource-allocation problem into a graph model.
- Gained knowledge of graph-coloring algorithms and chromatic-number concepts.
- Learned how to design and apply a backtracking search procedure.
- Understood the relationship between graph formulation and algorithmic solution quality.
- Improved understanding of applying graph theory to real-time engineering problems.

Engineering & Professional Skills Developed

- Graph-based modelling
- Algorithm design (graph coloring and backtracking)
- Logical reasoning and constraint satisfaction
- Python implementation
- Data organisation
- Technical documentation
- Testing and verification
- Engineering judgement

Individual Reflection

During the project, my main responsibility was the development of Module 2 – Graph-Based Parking Modeling using Graph Coloring and Backtracking. The most challenging part was designing a backtracking procedure that could correctly resolve conflicts without excessive computation time as the number of real-time requests grew. I implemented a graph-coloring routine that first attempts a fast, greedy assignment, and falls back to a systematic backtracking search only when a conflict is detected. I then verified every allocation by checking that no two adjacent (conflicting) slots received the same assignment. This work helped me understand the practical importance of combining efficient heuristics with a complete search procedure in real-time engineering applications. If the project were redesigned, I would further improve the model by incorporating heuristic vertex-ordering strategies (such as DSATUR) and real-time sensor feedback to reduce backtracking further.

REFERENCES

1. T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, Introduction to Algorithms, MIT Press.
2. N. Deo, Graph Theory with Applications to Engineering and Computer Science, Prentice Hall.
3. J. A. Bondy and U. S. R. Murty, Graph Theory, Springer.
4. D. Brelaz, New Methods to Color the Vertices of a Graph, Communications of the ACM.
5. S. Skiena, The Algorithm Design Manual, Springer.
6. Python Software Foundation, Python Documentation.
7. NetworkX Developers, NetworkX Documentation.
8. Matplotlib Development Team, Matplotlib Documentation.
9. M. Garey and D. Johnson, Computers and Intractability: A Guide to the Theory of NP-Completeness, W. H. Freeman.
10. R. K. Ahuja, T. L. Magnanti, and J. B. Orlin, Network Flows: Theory, Algorithms, and Applications, Prentice Hall.

.

APPENDIX A – DETAILED CALCULATIONS

. A.1 Parking Graph Representation

The formulated real-time parking model can be represented as an undirected graph:

$$\mathbf{G = (V, E)}$$

where $V = \{v_1, v_2, \ldots, v_n\}$ is the set of parking slots, and $E = \{(v_i, v_j) : \text{slot i conflicts with slot j}\}$ is the set of conflict relationships. The graph is stored using an adjacency list Adj[v] for efficient neighbour lookup during allocation.

A.2 Graph-Coloring with Backtracking – Pseudocode

```
FUNCTION allocateSlot(vertex v, graph G, colorAssignment):
   FOR each color c in AvailableColors:
      IF c is not used by any neighbour of v in colorAssignment:
         colorAssignment[v] = c
         RETURN TRUE
   RETURN FALSE   // no conflict-free color available


FUNCTION backtrackAllocate(vertexList, index, graph, colorAssignment):
   IF index == length(vertexList):
      RETURN TRUE   // all vertices successfully allocated
   v = vertexList[index]
   FOR each color c in AvailableColors:
      IF c is not used by any already-colored neighbour of v:
         colorAssignment[v] = c
         IF backtrackAllocate(vertexList, index + 1, graph, colorAssignment):
            RETURN TRUE
         REMOVE colorAssignment[v]   // backtrack: undo and try next color
   RETURN FALSE   // no valid assignment from this branch
```

This procedure is transferred to Module 2 for real-time processing of each new vehicle-arrival request against the parking graph produced in Module 1.

APPENDIX B – ENGINEERING DRAWING / FLOWCHART

Figure B.1 – Module 2 Flowchart (Graph Coloring & Backtracking)

START

↓

Receive Parking Graph G(V, E) from Module 1

↓

Receive New Real-Time Vehicle Request

↓

Select Candidate Slot (Vertex)

↓

Check Colors of Adjacent Vertices

↓

Conflict-Free Color Available?

↓

Yes → Assign Slot / Color to Vehicle | No → Backtrack to Previous Assignment

↓

Update Parking Status & Log Allocation

↓

More Vehicles in Queue? → Yes: Repeat | No: Continue

↓

END

APPENDIX J – INDIVIDUAL CONTRIBUTION EVIDENCE

The evidence for my individual contribution (Module 2) can include:

- Graph-coloring and backtracking design notes.
- Vertex / edge and color-assignment tables.
- Pseudocode and Python implementation of the allocation algorithm.
- Adjacency matrix / list used as algorithm input.
- Screenshots of allocation output and verification runs.
- Testing / verification records for conflict-free allocation.
- Version history or development screenshots.
- Team discussion / progress evidence related to Module 2.

The supplied capstone template specifically identifies Appendix J – Individual Contribution Evidence as mandatory for team projects.

CAPSTONE PROJECT OUTCOME MAPPING SHEET

To be completed by the department / faculty assessor.

Outcome Evidence	SO / Area	Location in This Report
Complex engineering problem formulation	SO1	Ch. 1, Ch. 2
Engineering design under constraints	SO2	Ch. 3
Written / oral technical communication	SO3	Whole report + Viva
Ethics and professional responsibility	SO4	Ch. 3 (3.7)
Teamwork and leadership	SO5	Ch. 4 (4.8)
Experimentation, analysis, engineering judgment	SO6	Ch. 4 (4.5–4.6)
Independent acquisition of new knowledge	SO7	Ch. 5 (5.3)
Standards	SO2	Ch. 3 (3.2)
Sustainability and societal / environmental impact	SO2/SO4	Ch. 3 (3.7)
Risk assessment and mitigation	SO2/SO4	Ch. 3 (3.7)
Security considerations	Relevant SO	Ch. 3 (3.7)
Integrated / system-level engineering	SO1/SO2	Ch. 3 (3.5)
Project planning and management	SO5	Ch. 4 (4.8)
Individual contribution	SO1–SO7	Contribution Statement + Ch. 4 (4.8)